

Visualizations in Jupyter Notebook (Python Data Analytics)

Matplotlib vs Seaborn vs Plotly

1. Matplotlib (Manual & Verbose)

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(8, 5))

# Must loop or plot categories manually for distinct colors
for cat, group in df.groupby('Category'):
    ax.scatter(group['Sales'], group['Profit'], label=cat, alpha=0.6)

ax.set_title("Sales vs. Profit by Category (Matplotlib)")
ax.set_xlabel("Sales ($)")
ax.set_ylabel("Profit ($)")
ax.legend()
plt.tight_layout()
plt.show()
```

2. Seaborn (Tidy-Data Native)

Handles grouping, coloring (hue), and aggregation automatically from DataFrame column names.

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x='Sales',
    y='Profit',
    hue='Category',
    alpha=0.6
)
plt.title("Sales vs. Profit by Category (Seaborn)")
plt.show()
```

3. Plotly Express (Interactive Out-of-the-Box)

Produces a fully interactive graphic where students can hover over individual points, zoom into dense areas, and double-click the legend to isolate categories.

```
import plotly.express as px

fig = px.scatter(
    df,
    x='Sales',
    y='Profit',
    color='Category',
    hover_data=['Sub-Category', 'Customer Name'],
```

```
    title="Sales vs. Profit by Category (Plotly)"
)
fig.show()
```

Matplotlib

Matplotlib uses an **Object-Oriented API** (fig, ax = plt.subplots()). This approach gives you full control over figure size, axes limits, tick formatters, and legends.

1. Dual-Axis Line Chart (Revenue & Margin Trend)

Compares two series on different scales over time using twinx().

```
monthly = df.groupby("YearMonth").agg(Sales=("Sales", "sum"), Profit=("Profit", "sum")).reset_index()

fig, ax1 = plt.subplots(figsize=(10, 4))

color_sales = "#1f77b4"
ax1.set_xlabel("Order Timeline")
ax1.set_ylabel("Gross Sales ($)", color=color_sales)
ax1.plot(monthly["YearMonth"], monthly["Sales"], color=color_sales, linewidth=2, label="Sales")
ax1.tick_params(axis="y", labelcolor=color_sales)

ax2 = ax1.twinx()
color_profit = "#2ca02c"
ax2.set_ylabel("Net Profit ($)", color=color_profit)
ax2.plot(monthly["YearMonth"], monthly["Profit"], color=color_profit, linewidth=1.5, linestyle="--",
label="Profit")
ax2.tick_params(axis="y", labelcolor=color_profit)

plt.title("Monthly Revenue vs. Net Profit Run-Rate")
plt.tight_layout()
plt.show()
```

2. Horizontal Ranked Bar Chart with Value Annotations

Ideal for displaying ranked categories with long labels.

```
subcat_sales = df.groupby("Sub-Category")["Sales"].sum().sort_values(ascending=True)

fig, ax = plt.subplots(figsize=(8, 6))
bars = ax.barh(subcat_sales.index, subcat_sales.values / 1000, color="#34495e", edgecolor="none")

# Add direct data callouts to eliminate axis-reading friction
for bar in bars:
    width = bar.get_width()
```

```

ax.text(width + 2, bar.get_y() + bar.get_height() / 2, f"${width:,.1f}K", va="center", fontsize=8)

ax.set_xlabel("Total Sales ($ in Thousands)")
ax.set_title("Ranked Revenue by Product Sub-Category")
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.show()

```

3. Stacked & Normalized 100% Bar Chart (Portfolio Mix)

Shows composition shifts across customer segments by region.

```

pivot_mix = pd.crosstab(df["Region"], df["Segment"], normalize="index") * 100

fig, ax = plt.subplots(figsize=(8, 5))
bottom = np.zeros(len(pivot_mix))
palette = ["#2b5c8f", "#d95f02", "#7570b3"]

for i, col in enumerate(pivot_mix.columns):
    values = pivot_mix[col].values
    ax.bar(pivot_mix.index, values, bottom=bottom, label=col, color=palette[i], width=0.55)
    bottom += values

ax.set_ylabel("Segment Share (%)")
ax.set_ylim(0, 100)
ax.set_title("Customer Segment Mix by Geographic Region")
ax.legend(loc="upper right", frameon=False)
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.show()

```

4. Frequency Histogram with Quantile Threshold Markers

Visualizes continuous distribution spread and highlights risk boundaries.

```

fig, ax = plt.subplots(figsize=(8, 4))
sales_capped = df[df["Sales"] < 1000]["Sales"]

n, bins, patches = ax.hist(sales_capped, bins=40, color="#4a90e2", edgecolor="white", alpha=0.85)

median_val = sales_capped.median()
p90_val = sales_capped.quantile(0.90)

ax.axvline(median_val, color="red", linestyle="--", linewidth=1.5, label=f"Median (${median_val:.1f})")
ax.axvline(p90_val, color="black", linestyle=":", linewidth=1.5, label=f"P90 Cutoff (${p90_val:.1f})")

ax.set_xlabel("Order Value ($)")

```

```

ax.set_ylabel("Transaction Count")
ax.set_title("Order Value Distribution (Truncated at $1,000)")
ax.legend(frameon=False)
plt.tight_layout()
plt.show()

```

5. Multi-Class Scatter with Dynamic Sizing (Bubble Plot)

Plots 4 dimensions at once: Sales (X), Profit (Y), Discount (Size), Category (Color).

```

fig, ax = plt.subplots(figsize=(9, 5))
category_colors = {"Technology": "#1f77b4", "Furniture": "#e67e22", "Office Supplies": "#27ae60"}

for cat, sub in df.groupby("Category"):
    ax.scatter(
        sub["Sales"],
        sub["Profit"],
        s=(sub["Discount"] * 250) + 15, # Size drives discount scale
        color=category_colors[cat],
        alpha=0.5,
        edgecolors="none",
        label=cat
    )

ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)
ax.set_xlim(-100, 10000)
ax.set_ylim(-3000, 4000)
ax.set_xlabel("Sales ($)")
ax.set_ylabel("Profit ($)")
ax.set_title("Transaction Exposure vs Profitability (Bubble Size = Concession Rate)")
ax.legend(frameon=True, loc="lower right")
plt.tight_layout()
plt.show()

```

6. Standard Financial Box-and-Whisker Plot

Evaluates five-number summaries across categories without statistical packages.

```

fig, ax = plt.subplots(figsize=(7, 5))

categories = df["Category"].unique()
grouped_data = [df[(df["Category"] == cat) & (df["Profit"].between(-200, 300))]["Profit"].values for cat in categories]

bp = ax.boxplot(grouped_data, labels=categories, patch_artist=True, notch=True)
colors = ["#ecf0f1", "#d5dbdb", "#bdc3c7"]

for patch, color in zip(bp["boxes"], colors):

```

```

    patch.set_facecolor(color)
for median in bp["medians"]:
    median.set(color="crimson", linewidth=2)

ax.set_ylabel("Profit Distribution Range ($)")
ax.set_title("Notched Profit Ranges Across Categories (Trimmed Outliers)")
plt.tight_layout()
plt.show()

```

7. Step Chart (Inventory Depletion / Fulfillment Squeeze)

Captures discrete state changes along time rather than smooth transitions.

```

cust_orders = df[df["Customer ID"] == "WB-21850"].sort_values("Order Date")

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.step(cust_orders["Order Date"], cust_orders["Sales"].cumsum(), where="post", color="#8e44ad",
        linewidth=2)
ax.scatter(cust_orders["Order Date"], cust_orders["Sales"].cumsum(), color="#2c3e50", zorder=3)

ax.set_xlabel("Timeline")
ax.set_ylabel("Cumulative Spend ($)")
ax.set_title("Single Account Spend Velocity Over Time (Discrete Jumps)")
plt.tight_layout()
plt.show()

```

8. Area Chart / Running Balance Drawdown

Displays cumulative volume build-up with solid baseline fills.

```

monthly_loss = (
    df[df["Profit"] < 0]
    .resample("ME", on="Order Date")["Profit"]
    .sum()
    .abs()
    .cumsum()
)

fig, ax = plt.subplots(figsize=(9, 4))
ax.fill_between(monthly_loss.index, monthly_loss.values, color="#e74c3c", alpha=0.3)
ax.plot(monthly_loss.index, monthly_loss.values, color="#c0392b", linewidth=2)

ax.set_ylabel("Cumulative Dollar Loss ($)")
ax.set_title("Historical Portfolio Cumulative Charge-Off Curve")
plt.tight_layout()
plt.show()

```

Seaborn

Seaborn natively accepts tidy/long-form Pandas DataFrames, handling grouping, statistical aggregation, confidence interval computation, and color palettes automatically.

```
# Apply standard clean aesthetic theme
sns.set_theme(style="ticks", palette="muted")
```

1. Regression Plot with Confidence Intervals (sns.regplot)

Measures linear elasticity between discount concession rates and resulting profit margins.

```
sample_df = df.sample(n=1000, random_state=42).copy()
sample_df["Profit_Margin"] = (sample_df["Profit"] / sample_df["Sales"]) * 100

plt.figure(figsize=(8, 5))
sns.regplot(
    data=sample_df[sample_df["Profit_Margin"].between(-100, 100)],
    x="Discount",
    y="Profit_Margin",
    scatter_kws={"alpha": 0.25, "color": "#2980b9", "s": 20},
    line_kws={"color": "#c0392b", "linewidth": 2}
)
plt.title("Discount-to-Margin Elasticity Curve (with 95% Bootstrap CI)")
plt.ylabel("Profit Margin (%)")
plt.xlabel("Discount Applied (0.0 to 1.0)")
sns.despine()
plt.show()
```

2. Categorical Violin Plot with Inner Quartiles (sns.violinplot)

Combines box-plot metrics with a full Kernel Density Estimation (KDE) to show multimodal clustering.

```
plt.figure(figsize=(9, 5))
sns.violinplot(
    data=df[df["Sales"] < 600],
    x="Category",
    y="Sales",
    hue="Segment",
    split=False,
    inner="quart",
    palette="Blues"
)
plt.title("Transaction Size Density Distribution Across Segments")
plt.xlabel("Merchandise Category")
plt.ylabel("Sales ($)")
sns.despine()
plt.show()
```

3. Diverging Correlation Heatmap (sns.heatmap)

Isolates multicollinear risk variables using custom diverging colormaps and masks.

```
numeric_metrics = df[["Sales", "Quantity", "Discount", "Profit"]].corr()

plt.figure(figsize=(6, 5))
mask = np.triu(np.ones_like(numeric_metrics, dtype=bool)) # Mask upper triangle

sns.heatmap(
    numeric_metrics,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap="vlag",
    vmin=-1.0,
    vmax=1.0,
    linewidths=1,
    cbar_kws={"shrink": 0.8}
)
plt.title("Financial Factor Correlation Matrix")
plt.show()
```

4. Bivariate Joint Distribution with Hexbins (sns.jointplot)

Addresses point overplotting by aggregating overlapping transactions into 2D density bins.

```
clean_subset = df[(df["Sales"].between(10, 1500)) & (df["Profit"].between(-200, 400))]

g = sns.jointplot(
    data=clean_subset,
    x="Sales",
    y="Profit",
    kind="hex",
    color="#16a085",
    gridsize=30,
    height=6
)
g.fig.subplots_adjust(top=0.9)
g.fig.suptitle("Hexbin Joint Density of Transactions: Sales vs. Profit")
plt.show()
```

5. Multi-Distribution Kernel Density Plot (sns.kdeplot)

Compares smooth, continuous probability density functions across categories without binning noise.

```
plt.figure(figsize=(8, 4.5))
sns.kdeplot(
    data=df[df["Sales"].between(5, 500)],
    x="Sales",
    hue="Category",
    fill=True,
    common_norm=False,
    alpha=0.25,
    linewidth=2
)
plt.title("Comparative Sales Density Curves by Category (Under $500)")
plt.xlabel("Transaction Volume ($)")
sns.despine()
plt.show()
```

6. Aggregated Point Plot with Bootstrap Error Bands (sns.pointplot)

Highlights interaction effects and relative differences in means rather than full distributions.

```
plt.figure(figsize=(8, 5))
sns.pointplot(
    data=df,
    x="Region",
    y="Discount",
    hue="Segment",
    dodge=True,
    markers=["o", "s", "D"],
    linestyle=["-", "--", ":"],
    capsize=0.1
)
plt.title("Mean Concession Rates Across Geographies (with CI error bars)")
plt.ylabel("Mean Discount Rate")
sns.despine()
plt.show()
```

7. Categorical Strip Plot with Jitter (sns.stripplot)

Exposes raw transaction dots alongside categorical levels to reveal underlying sample size.

```
plt.figure(figsize=(8, 5))
sns.stripplot(
    data=df[df["Profit"] < -200],
    x="Region",
    y="Profit",
    hue="Category",
    jitter=0.25,
    size=6,
    alpha=0.7,
    palette="Dark2"
)
plt.title("Severe Loss Outliers (< -$200) by Geographic Region")
plt.ylabel("Net Loss Amount ($)")
sns.despine()
plt.show()
```

8. Small-Multiples Trellis Grid (sns.FacetGrid / sns.catplot)

Builds a matrix of identical charts split across discrete dimensions.

```
# Multi-panel visualization: Rows = Segment, Columns = Category
g = sns.catplot(
    data=df,
    x="Ship Mode",
    y="Profit",
    col="Category",
    row="Segment",
    kind="bar",
    estimator=np.mean,
    errorbar=None,
    height=2.5,
    aspect=1.4,
    palette="ch:s=0.2,r=0.6"
)

g.set_axis_labels("", "Mean Profit ($)")
g.set_xticklabels(rotation=45, ha="right")
g.fig.subplots_adjust(top=0.92)
g.fig.suptitle("Mean Profitability Across 9 Segments & Categories")
plt.show()
```

Animated and 3D Charts

Inside a Jupyter Notebook, to see animations render smoothly inline as interactive HTML5 video players rather than static snapshots, use this one-line setup:

Connect to dataset

```
%matplotlib inline
import matplotlib.pyplot as plt
from matplotlib import animation
from mpl_toolkits.mplot3d import Axes3D
from IPython.display import HTML
import pandas as pd
import numpy as np

# Load dataset and prepare chronological aggregation
df = pd.read_csv("Sample - Superstore.csv", encoding="windows-1252")
df["Order Date"] = pd.to_datetime(df["Order Date"])
df["YearMonth"] = df["Order Date"].dt.to_period("M").dt.to_timestamp()
```

3D Visualizations (mpl_toolkits.mplot3d)

3D charts require passing `projection="3d"` into `fig.add_subplot()` or `plt.subplots()`. You can adjust the camera orientation using `ax.view_init(elev=..., azim=...)`.

1. 3D Risk & Margin Scatter Plot

Maps 3 continuous metrics: Sales (X), Discount (Y), and Profit (Z) with dynamic color-coding.

```
fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection="3d")

sample = df.sample(n=800, random_state=42)

p = ax.scatter(
    sample["Sales"],
    sample["Discount"],
    sample["Profit"],
    c=sample["Profit"],
    cmap="coolwarm_r",
    s=25,
    alpha=0.6,
    edgecolors="none"
)
```

```

ax.set_xlabel("Sales ($) ", labelpad=10)
ax.set_ylabel("Discount Rate", labelpad=10)
ax.set_zlabel("Profit ($) ", labelpad=10)
ax.set_title("3D Exposure Surface: Sales vs. Discount vs. Profit")
ax.view_init(elev=20, azim=135)

fig.colorbar(p, ax=ax, shrink=0.5, pad=0.1, label="Profit ($)")
plt.tight_layout()
plt.show()

```

2. 3D Bivariate Bar Chart (Regional Category Matrix)

Projects a 2D cross-tabulation onto discrete 3D columns.

```

pivot_data = pd.crosstab(df["Region"], df["Category"], values=df["Sales"], aggfunc="sum") / 1000

fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection="3d")

x_labels = pivot_data.columns.tolist() # Categories
y_labels = pivot_data.index.tolist() # Regions

_x = np.arange(len(x_labels))
_y = np.arange(len(y_labels))
_xx, _yy = np.meshgrid(_x, _y)
x_pos, y_pos = _xx.ravel(), _yy.ravel()
z_pos = np.zeros_like(x_pos)

dx = dy = 0.45
dz = pivot_data.values.ravel()

# Color bars by height
colors = plt.cm.viridis(dz / dz.max())

ax.bar3d(x_pos, y_pos, z_pos, dx, dy, dz, color=colors, shade=True)

ax.set_xticks(_x + dx / 2)
ax.set_xticklabels(x_labels, rotation=15, ha="right")
ax.set_yticks(_y + dy / 2)
ax.set_yticklabels(y_labels)
ax.set_zlabel("Sales ($ in Thousands)")
ax.set_title("3D Regional Product Sales Volumes")
ax.view_init(elev=25, azim=45)

plt.tight_layout()
plt.show()

```

3. Continuous 3D Parametric Loss Surface (plot_surface)

Interpolates an aggregated grid to visualize the structural relationship between transaction size, discounts, and expected net return.

```
# Generate a regular 2D meshgrid of Discount and Sales
discount_grid = np.linspace(0.0, 0.8, 25)
sales_grid = np.linspace(100, 2000, 25)
D, S = np.meshgrid(discount_grid, sales_grid)

# Theoretical expected profit formula: Margin degrades exponentially as discount increases
P = S * (0.25 - (1.1 * D))

fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection="3d")

surf = ax.plot_surface(S, D, P, cmap="Spectral", edgecolor="none", alpha=0.85)

ax.set_xlabel("Transaction Size ($)"), labelpad=10)
ax.set_ylabel("Discount Rate", labelpad=10)
ax.set_zlabel("Estimated Profit ($)"), labelpad=10)
ax.set_title("Estimated Profitability Response Surface")
ax.view_init(elev=22, azimuth=-60)

fig.colorbar(surf, ax=ax, shrink=0.5, pad=0.1)
plt.tight_layout()
plt.show()
```

Animated Charts (matplotlib.animation.FuncAnimation)

The standard animation workflow uses **FuncAnimation**:

1. Create a figure and axis.
2. Initialize empty line/scatter objects (init() function).
3. Define an update(frame) function that mutates the data for each step.
4. Render the output inside Jupyter using HTML(anim.to_jshtml()).

1. Animated Portfolio Timeline (Moving Time-Series Line)

Draws monthly sales run-rate progressively across time.

```
from matplotlib import animation
from IPython.display import HTML

df["YearMonth"] = df["Order Date"].dt.to_period('M')
monthly_sales = df.groupby("YearMonth")["Sales"].sum().reset_index()

# Convert Period objects to datetime for matplotlib compatibility
```

```

monthly_sales["YearMonth_dt"] = monthly_sales["YearMonth"].dt.to_timestamp()

fig, ax = plt.subplots(figsize=(8, 4))
# Use the datetime version for axis limits
ax.set_xlim(monthly_sales["YearMonth_dt"].min(), monthly_sales["YearMonth_dt"].max())
ax.set_ylim(0, monthly_sales["Sales"].max() * 1.1)
ax.set_ylabel("Billed Revenue ($)")
ax.set_title("Animated Monthly Sales Progression")
plt.xticks(rotation=30)

line, = ax.plot([], [], color="#2b5c8f", linewidth=2.5, marker="o", markersize=4)

def init():
    line.set_data([], [])
    return line,

def update(frame):
    current_data = monthly_sales.iloc[:frame+1]
    # Use the datetime version for plotting
    line.set_data(current_data["YearMonth_dt"], current_data["Sales"])
    return line,

anim = animation.FuncAnimation(
    fig, update, frames=len(monthly_sales), init_func=init, blit=True, interval=120
)
plt.close() # Suppress duplicate static plot display

# Render inside Jupyter
HTML(anim.to_jshtml())

```

2. Animated Bar Chart Race (Cumulative Regional Sales)

Visualizes rank swaps and accelerating cumulative volume across regions over time.

```

from matplotlib import animation
from IPython.display import HTML
from datetime import datetime
import matplotlib.pyplot as plt
# Cumulative monthly sales by region

# df["YearMonth"] = df["Order Date"].dt.year.to_string() + df["Order Date"].dt.month_name().to_string()
df["YearMonth"] = df["Order Date"].dt.to_period('M')
cu_region = (
    df.pivot_table(index="YearMonth", columns="Region", values="Sales", aggfunc="sum")
    .fillna(0)
    .cumsum()
)

```

```

fig, ax = plt.subplots(figsize=(8, 4.5))
regions = cu_region.columns.tolist()
colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]

def updateBars(frame):
    ax.clear()
    current_date = cu_region.index[frame]
    current_vals = cu_region.iloc[frame]

    # Sort for bar racing effect
    sorted_idx = current_vals.sort_values().index
    sorted_vals = current_vals[sorted_idx]

    bars = ax.barh(sorted_idx, sorted_vals / 1000, color=colors)
    ax.set_xlim(0, cu_region.values.max() / 1000 * 1.15)
    ax.set_xlabel("Cumulative Sales ($ in Thousands)")
    ax.set_title(f"Regional Race - As of {current_date.to_timestamp().strftime('%B %Y')}", fontsize=11)

    # Callout annotations
    for bar in bars:
        w = bar.get_width()
        ax.text(w + 3, bar.get_y() + bar.get_height() / 2, f"${w:,.1f}K", va="center", fontsize=9)
    ax.spines[["top", "right"]].set_visible(False)

anim_race = animation.FuncAnimation(fig, updateBars, frames=len(cu_region), interval=150)
plt.close()

HTML(anim_race.to_jshtml())

```

3. Dynamic Bubble Trail / Migration Plot

Simulates the classic "Gapminder" style: Category bubbles moving across the Sales/Profit coordinate space over successive quarters.

```

from matplotlib import animation
from IPython.display import HTML

quarterly = (
    df.groupby([df["Order Date"].dt.to_period("Q"), "Category"])
    .agg(Sales=("Sales", "mean"), Profit=("Profit", "mean"), Count=("Sales", "count"))
    .reset_index()
)

quarters = quarterly["Order Date"].unique()
categories = quarterly["Category"].unique()

```

```

cat_palette = {"Furniture": "#e67e22", "Office Supplies": "#27ae60", "Technology": "#2980b9"}

fig, ax = plt.subplots(figsize=(8, 5))
ax.set_xlim(0, 600)
ax.set_ylim(-50, 150)
ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)
ax.set_xlabel("Average Ticket Size ($)")
ax.set_ylabel("Average Profit ($)")

def update_bubbles(frame):
    ax.clear()
    ax.set_xlim(0, 600)
    ax.set_ylim(-50, 150)
    ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)
    ax.set_xlabel("Average Ticket Size ($)")
    ax.set_ylabel("Average Profit ($)")

    q_current = quarters[frame]
    sub = quarterly[quarterly["Order Date"] == q_current]

    ax.set_title(f"Category Performance Dynamics - {q_current}", fontsize=11)

    for _, row in sub.iterrows():
        ax.scatter(
            row["Sales"],
            row["Profit"],
            s=row["Count"] * 1.2,
            color=cat_palette[row["Category"]],
            alpha=0.75,
            label=row["Category"]
        )
    ax.annotate(row["Category"], (row["Sales"] + 10, row["Profit"]))
    ax.spines[["top", "right"]].set_visible(False)

anim_bubbles = animation.FuncAnimation(fig, update_bubbles, frames=len(quarters), interval=300)
plt.close()

HTML(anim_bubbles.to_jshtml())

```